

When Fancy Eviction Fails: Rethinking Cache Replacement For LLM Prefix Reuse

Yiyu Liu
Harvard University

Minlan Yu
Harvard University

Juncheng Yang
Harvard University

Abstract

Long-running LLM applications repeatedly send growing context, making prefix caching critical for reducing prefill cost. Yet prefix-cache behavior under agentic workloads remains poorly understood. We study production traces from two organizations and evaluate 14 eviction algorithms across HBM-constrained and large memory-pool settings. Despite a large gap to Belady, sophisticated policies designed for traditional caches provide little benefit over LRU. The reason is structural: prefix reuse is dominated by the regular pacing of active sessions, making recency unusually predictive. Prefix caching nevertheless introduces new challenges, including heavy-tailed session footprints and highly variable miss costs as attention computation grows with sequence length. We introduce the compute-savings ratio and two offline oracles to quantify these effects. Our results show that effective prefix-cache management should retain recency as its foundation while selectively adding quick demotion for one-hit prefixes, compute-aware partial eviction for expensive misses, and capacity-dependent eviction granularity. We will release the traces and simulator to support future research.

1 Introduction

Long-running conversations and autonomous agents are rapidly pushing LLM context lengths into the hundreds of thousands of tokens [49, 50, 59, 64, 73]. Recomputing these contexts is increasingly expensive because Transformer prefill cost grows rapidly with sequence length [27, 61]. Fortunately, successive requests within a session typically reuse a long prefix [23]. Modern serving systems therefore rely on *prefix caching*: they retain the key-value (KV) states of previously processed tokens and reuse them across requests, avoiding redundant prefill computation [29, 51, 76]. As contexts grow, the efficiency of this cache becomes an increasingly important determinant of LLM serving cost and performance.

Traditional cache workloads, including web content and block storage, have been studied extensively [8, 10, 28, 32],

yielding a rich set of eviction algorithms [7, 15, 19, 44, 45, 74]. LLM serving workloads are much less understood. Recent work has begun to characterize aggregate LLM traffic [3, 38, 64, 79], but the access patterns that determine prefix-cache eviction, particularly for agentic workloads, remain largely unexplored. We address this gap using production traces and ask a basic question: *what makes a good eviction policy for LLM prefix caches?*

We first test whether decades of cache-management techniques transfer directly. Most LLM serving systems use Least Recently Used (LRU) eviction [29, 40, 51, 76], both in capacity-constrained *per-replica HBM* (24–120 GiB per GPU) and in much larger *global memory pools* (0.25–12 TiB) [40, 51, 68]. In traditional caches, algorithms that combine recency, frequency, and learned signals can substantially outperform LRU [5, 18, 26, 45, 57, 62, 72, 74, 77]. We evaluate 14 advanced algorithms [56, 64] across both memory regimes and find a surprising result: most offer little improvement over LRU, while frequency-based algorithms often perform substantially worse. At the same time, Belady’s optimum [6] shows substantial remaining headroom.

This gap suggests that prefix-cache workloads differ fundamentally from the workloads these algorithms were designed for. Understanding why LRU is so difficult to beat—and what signals remain exploitable beyond recency—is therefore the key to designing better prefix-cache management.

Comparing prefix-cache traces with the web and block-storage workloads that drove conventional eviction policies reveals the source of this mismatch. Most prefix reuse comes from successive turns within the same session. As a result, cached blocks have short, session-bounded lifetimes, and reuse intervals are tightly concentrated. Recency captures this structure naturally. Frequency, in contrast, largely reflects how long a session has been active and provides little information about when its blocks will be reused. This explains both LRU’s robustness and the poor performance of frequency-based policies.

Prefix caching also departs from traditional caching in two important ways. First, miss costs are highly non-uniform [56]:

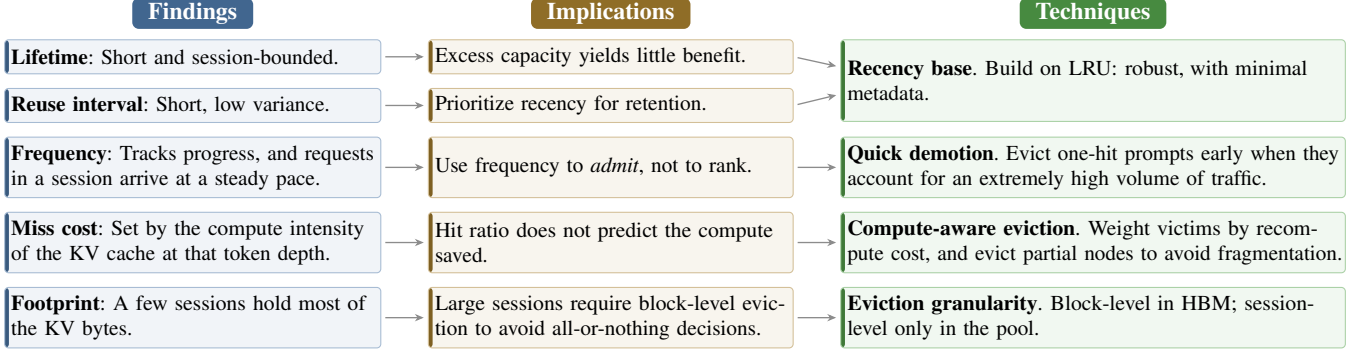


Figure 1: Paper overview: We characterize five fundamental properties of production prefix-cache workloads (§4), derive their implications for cache management, and map them to our targeted design techniques (§5).

because attention cost increases with token position, reconstructing a block deep in a prompt is more expensive than reconstructing an equally sized block near the beginning. Thus, two policies with the same hit ratio can incur very different prefill costs and time-to-first-token (TTFT). Second, session KV footprints are highly skewed, with a small number of large sessions consuming a disproportionate fraction of cache capacity and contributing to a large portion of hits (i.e., size correlated with frequency). Figure 1 summarizes these properties and their implications for cache design.

These observations suggest a simple principle: keep recency as the default, and add signals only when the workload requires them. We find that *quick demotion* [72] helps primarily when one-hit prompts are common. For non-uniform miss costs, we derive an offline optimum and an efficient approximation, showing that compute-aware eviction can improve performance but must control fragmentation. Finally, the right eviction granularity depends on cache capacity: constrained HBM benefits from block-level management, whereas large memory pools can use session-level eviction to reduce metadata overhead with little efficiency loss.

This paper makes the following main contributions:

- **The first production study of prefix-cache eviction.** We introduce two production prefix-cache traces and systematically evaluate a broad set of eviction policies across both HBM and memory-pool regimes, validating our conclusions on four additional Qwen Bailian workloads [64].
- **A faithful, high-performance prefix-cache simulator.** We build a C++ simulator that captures request-level residency constraints imposed by real LLM serving engines, enabling realistic large-scale eviction studies.
- **A workload explanation for why LRU is hard to beat.** By contrasting prefix-cache traces with canonical web and block-storage workloads, we identify the structural properties that make recency unusually predictive and render many traditional eviction signals ineffective.
- **A compute-aware objective and offline bounds.** We introduce the *compute-savings ratio* to capture the highly non-uniform cost of prefix-cache misses, and derive an

exact offline optimum and an efficient approximation that bound achievable performance.

- **New design principles for prefix-cache management.** We show when and how to augment recency with quick demotion, compute-aware eviction, and capacity-adaptive eviction granularity, yielding concrete guidance for both constrained HBM caches and large global memory pools.
- **Open traces and tools.** We will release our production traces and simulator to enable reproducible research on prefix-cache management.

2 Background

2.1 LLM Inference

Each Transformer layer [61] executes three core operations: QKVO projections, attention, and a Multi-Layer Perceptron (MLP). Linear QKV projections map hidden states into Query, Key, and Value representations. The attention mechanism then computes query-key dot products to aggregate these values, capturing sequence dependencies before an Output projection. Finally, the MLP applies non-linear transformations to refine each token’s representation.

2.2 LLM KV Cache

To eliminate redundant attention computation during autoregressive generation, LLM inference systems rely on a *KV cache* [29]. This cache bridges the two core phases of inference: *prefill* and *decode*. During the prefill phase, the model processes the entire prompt, saving the resulting key and value vectors to GPU memory. The latency of this initial processing, including any queuing delay, is measured as Time To First Token (TTFT). In the subsequent decode phase, the model generates tokens sequentially, attending to the KV cache built during the prefill phase rather than recomputing them. The speed of this generation is measured as the Time Per Output Token (TPOT). Because the KV cache must preserve the states of all active tokens, it rapidly becomes a severe

memory bottleneck as context lengths and batch sizes grow, fundamentally constraining serving capacity [29].

2.3 LLM Serving Prefix Cache

In many real-world workloads, requests frequently share common prefixes, such as lengthy system prompts [49, 50], few-shot examples [11], and, in multi-turn conversations and agent loops, the whole history of the exchange so far [73].

Recomputing these shared prefixes for every request is highly inefficient because the computational cost of the prefill phase grows rapidly. As context lengths expand to support document question answering [4, 39] and autonomous agents [49, 73], the compute burden becomes massive. To make matters worse, for the full exact self-attention layers, the computation scales quadratically with sequence length [27, 61]. The rise of Mixture-of-Experts (MoE) models [55, 59, 78] exacerbates this quadratic bottleneck by sparsifying MLP computations, causing attention to dominate the prefill phase and making cache hits even more valuable.

To eliminate this massive redundant computation, serving systems cache and reuse previously computed KV states. However, due to the causal masking inherent in Transformer decoders, the KV state of a token depends on all tokens that precede it. Consequently, a cached state can be safely reused if and only if its entire preceding sequence exactly matches the new request. This strict sequential dependency naturally shapes the cache’s architecture, organizing the reusable KV states into a prefix tree (or a memory-efficient radix tree, as popularized by SGLang [76]).

Recognizing these substantial benefits, modern LLM serving systems have widely adopted this *prefix caching* technique. In fact, production deployments report that a large portion of their total traffic is now served directly from these caches [51, 64]. These systems efficiently manage KV states by chunking them into fixed-length blocks (typically 16 tokens per block [29]). To maximize hit rates, they typically deploy prefix caches across a two-tiered memory hierarchy. At the first tier, the serving engine keeps KV blocks in the GPU High-Bandwidth Memory (HBM) [29, 76]. At the second tier, systems like Mooncake [51] and Strata [68] introduce a massive global pool backed by host DRAM and SSDs. This shared pool is one to two orders of magnitude larger than the local HBM, trading the overhead of cross-device data transfers for vastly increased capacity.

While the prefix tree provides an effective logical data structure for organizing and matching these shared KV blocks, it does not dictate physical cache eviction. As long as the system can determine that a cached block shares exactly the same prefix as the new request, that block can be safely reused. Hence, physical cache eviction is fundamentally decoupled from the logical prefix tree structure, allowing algorithms to drop blocks from anywhere in the cache.

Table 1: LLM inference request traces from two organizations. Requests are capped at 256K tokens.

	FreeInference Trace	Chutes Trace
Requests	327.5 K	515.8 K
in multi-turn sessions	34.3%	26.1%
Span	7.0 d	130.9 d
Tokens processed	10.5 B	9.7 B
Unique tokens	0.63 B	2.23 B
Request tokens, average	32.0 K	18.7 K
99th percentile	219.3 K	98.3 K

Table 2: Cache sizes we evaluate.

Size	Blocks	Example setting
<i>HBM, per replica: 24–120 GiB</i>		
24 GiB	16,384	native-context floor
48 GiB	32,768	1×H200 at 0.8 memory fraction
96 GiB	65,536	1×B200, bf16 weights
<i>DRAM/SSD pool, global: 0.25–12 TiB</i>		
1 TiB	699,051	full node in Mooncake [51]

2.4 Sessions

A *session* represents a single continuous conversation [13] or autonomous agent task [49, 73]. It comprises sequential requests (*turns*) that progressively append to a shared context. While engines schedule individual requests, the session remains the logical unit of interaction.

This append-only structure preserves history and maximizes prefix cache reuse. Each request appends new information, such as user inputs or tool outputs, to an earlier turn. This iterative loop, where a model dispatches a tool call, the runtime executes it, and the output is seamlessly woven back into the context for the next turn, forms the agent workflow.

2.5 Traditional Web and Storage Cache

Traditional caching research typically focuses on workloads derived from block storage systems and web (e.g., content delivery networks). These workloads exhibit distinct access patterns. For instance, block I/O traces are frequently dominated by sequential scans, while web traffic is characterized by Zipfian popularity distributions and a massive volume of one-hit wonders. Decades of eviction algorithms have been heavily optimized for these specific behaviors.

3 Fancy Algorithms Do Not Work Well

To understand the unique character of LLM prefix cache, we begin by evaluating how state-of-the-art caching algorithms perform on production traces. We first establish two practical cache-size regimes for evaluation (§3.1), and then directly compare fourteen online eviction algorithms against

Table 3: Comparison of public LLM-serving traces with the two traces introduced in this work. Only FreeInference Trace and Chutes Trace combine fine-grained block identity scoped across sessions, agentic traffic, and a multi-day span.

Trace	Requests	Tokens	Span	Block size	Scope	Agentic
TraceLab [79] [†]	357 K	54.9 B	253 d, scattered	none	—	✓
Mooncake, tool & agent [51]	23.6 K	0.20 B	1 h	coarse (512 tokens)	global	✓
AgentX [53]	98.8 K	21.6 B	per session	intermediate (64 tokens)	per session	✓
Qwen Bailian [64]*	10.8–172.8 K	0.05–0.25 B	2 h	fine (16 tokens)	global	very little
FreeInference Trace (this work)	327.5 K	10.5 B	7.0 d	fine (16 tokens)	global	✓
Chutes Trace (this work)	515.8 K	9.7 B	130.9 d	fine (16 tokens)	global	✓

[†]Records are LLM interactions from 43 developers’ individual sessions (1.4 K rounds/day over eight months), not a continuous production service stream.

*The dataset comprises four independent traces, each spanning a two-hour window with its own localized clock.

the optimal Belady’s oracle across two production serving traces (§3.2). Our results demonstrate that most complex algorithms yield little benefit over a simple recency-based baseline (LRU), and in some cases, actively degrade performance.

3.1 Traces and Simulator

Traces. We conduct our evaluation using two production traces from two different public LLM inference services: FreeInference Trace [22] and Chutes Trace [16]. FreeInference Trace is heavily dominated by agentic workloads with some non-agentic API usage, while Chutes Trace features a broader mix that includes multi-turn human conversations alongside agentic sessions and non-agentic API usage. To align with practical serving constraints, all evaluations use a 16-token block size and the Qwen3-Coder-30B [59] tokenizer, capping sequences at its 256k-token maximum context window (4.3% skipped in FreeInference Trace and 0.03% in Chutes Trace). Table 1 summarizes the key characteristics of each trace. Table 3 compares existing public datasets with the two traces introduced in this work. To accurately simulate prefix-cache eviction, a trace must provide fine-grained block identity scoped across sessions and contain multi-turn agentic traffic. Existing public traces lack at least one of these critical properties. The closest alternatives are the Qwen Bailian traces, which we also evaluate (§4.6). While they offer 16-token block hashes, their session information reveals very little agentic workload and a much lower reuse ratio.

Simulator. To evaluate caching algorithms over these traces, standard block-level simulators like libCacheSim [71] are insufficient because they ignore the request-level residency restriction, which requires that all blocks for a request must reside in memory simultaneously during its prefill. To faithfully replicate real-world engine behavior, we developed a custom, high-performance C++ simulator based on libCacheSim. Our simulator proactively evicts blocks to ensure sufficient space is available to accommodate an incoming request’s entire prefix at once, and its evaluated hit ratios are very close to those of the native vLLM [29] engine. Compared to the vLLM’s python implementation, this simulator yields up to

160× speedup in evaluation.

Cache-size regimes. While memory per block varies by model, block-level eviction dynamics remain consistent and the required capacity simply scales accordingly. We evaluate eviction algorithms across two distinct capacity regimes, summarized in Table 2. First, the *per-replica HBM* regime models the constrained memory available directly on accelerators, ranging from 24 GiB to 120 GiB. Scaling below 24 GiB is impractical, as the residency constraint would severely restrict the maximum supported context length. Second, the *global DRAM/SSD memory pool* regime models massive disaggregated storage, spanning from 0.25 TiB to 12 TiB.

3.2 Evaluation of Eviction algorithms

With our traces and simulator established, we sweep both cache-size regimes to evaluate how various caching mechanisms perform under a uniform cost model. We report hit and compute-savings ratios as fractions, and differences between them in percentage points.

Figure 2 shows the hit ratios of 14 online eviction algorithms across both production traces. We group these algorithms by their primary design principle: *Recency*, acting as our baseline (LRU); *Quick demotion*, which aggressively evicts unproven blocks (ARC [45], Sieve [74], S3-FIFO [72], S4-FIFO [66], LIRS [26]); *Analytic modeling*, using fitted mathematical models (LHD [5]); *Frequency* (LFU, W-TinyLFU [18]); *Learned* algorithms (LeCaR [62], LRB [57], 3LCache [77]); and recent *domain-specific* prefix caching algorithms (Workload-aware [64], AsymCache [56]). Finally, we use Belady’s optimum [6] to establish an *offline ceiling* for comparison.

Our evaluation reveals that LRU remains remarkably robust across prefix caching workloads, while more complex designs yield little benefit, or even degrade performance. Rather than outperforming this simple baseline, the advanced algorithms fall into three behavioral groups. First, a large majority of algorithms, including ARC, LHD, LeCaR, 3LCache, LRB, Workload-aware and AsymCache, perform very similar, or slightly worse to LRU. Despite their sophisticated

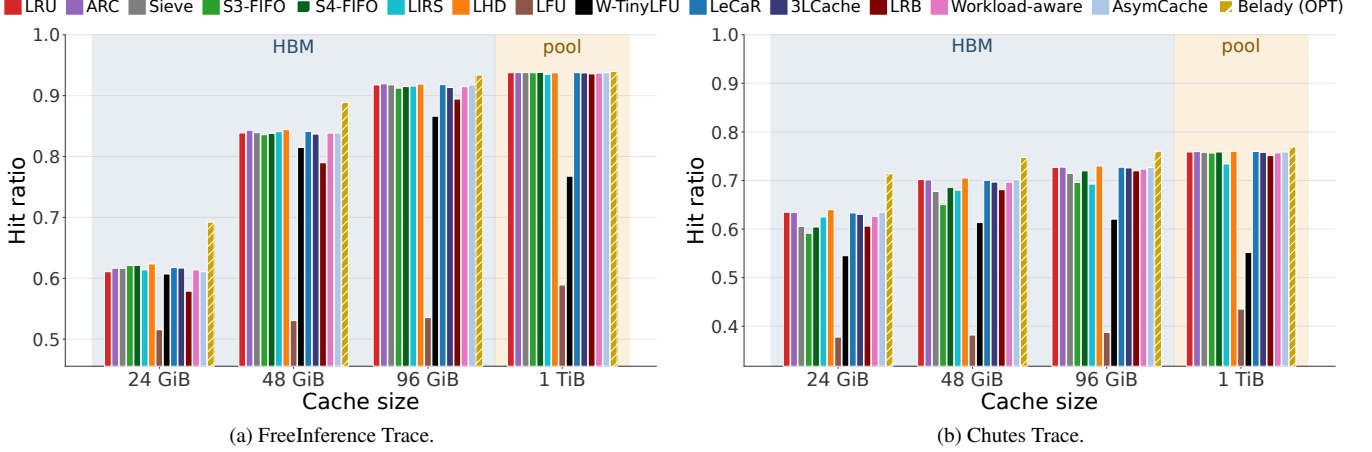


Figure 2: Block hit ratio on the two production traces for fourteen eviction algorithms and offline optimum (Belady). Across different memory capacities, none of the state-of-the-art algorithms improve over LRU.

Table 4: We compare three different caching workloads in this paper: LLM serving prefix cache, web cache (CDN), and block cache.

Class	Trace	Requests	Distinct items	Span	Description
Prefix cache	FreeInference Trace	327.5 K	39.6 M blocks	7.0 d	One week of a public LLM inference service.
Web	Wikipedia 2019 text [12]	207.6 M	18.4 M objects	21.0 d	Wikipedia’s CDN request log for article text.
Block	CloudPhysics w01 [12, 63]	215.8 M	122.8 M objects	6.3 d	Disk I/O of one VM volume from a production fleet.

designs, they merely reproduce LRU’s hit ratio without providing any meaningful advantage. Second, quick demotion algorithms (S3-FIFO, S4-FIFO, Sieve, LIRS) prove highly trace-dependent, offering marginal gains on some workloads but regressing by several points on others. Third, frequency-based algorithms, including LFU and W-TinyLFU, completely collapse, trailing far behind. Ultimately, there remains a significant headroom: Belady’s offline oracle still sits far above the best online algorithm. No current “fancy” mechanism fundamentally solves the eviction problem for prefix caching.

4 What Makes Prefix-Cache Different

Traditional caching algorithms target web CDNs and block I/O, where objects are drawn from persistent global pools. Prefix caching [29, 76], however, differs fundamentally. Instead of drawing from static pools, its blocks accumulate dynamically through conversations and tool executions, creating distinct access behaviors. To illustrate this, we compare our prefix-cache workload (FreeInference Trace from §3) with two canonical traces representing the primary workload classes in eviction-algorithm literature [5, 15, 72]: Wikipedia [12] (web CDN) and CloudPhysics [12, 63] (production block I/O). Table 4 summarizes these three workloads.

We treat each KV block [61] as a distinct cache object. The following subsections detail the key characteristics distinguishing these workloads (summarized in Table 5) and identify their root causes.

4.1 Session-Bounded Lifetimes

Prefix-cache objects exhibit drastically shorter lifespans than those in traditional caching workloads. Figure 3a illustrates this contrast: while traditional objects drawn from persistent pools remain active for days, *most reused prefix-cache objects exhibit lifetimes under 30 minutes*.¹ Because these blocks are generated by conversations, their utility is usually bounded by the originating session’s lifespan. Figure 3b further illustrates that multi-turn sessions are predominantly short-lived (median 55 s, 99th percentile ~4 hours), imposing a natural ceiling on the lifespans of most blocks. Notably, the proportion of one-hit objects (lifetime of zero in Figure 3a) is similar across all three workloads (53.0%–55.7%).

Across the workload, prefix-cache blocks naturally divide into three categories with distinct lifecycle dynamics:

System prompts. This category consists of system-role blocks appearing in at least one multi-turn sessions. Despite comprising only 4.7% of distinct blocks, their cross-session reuse drives 18.4% of all accesses. They routinely outlive individual conversations, persisting in the cache for days (median lifetime 291 s, 99th percentile ~3.6 days).

Multi-turn session history. This category includes all non-system blocks that appear in at least one multi-turn session. Representing session-specific state like conversational turns and tool outputs, these blocks constitute the primary driver of prefix reuse. They make up 37.6% of distinct blocks and generate an overwhelming 70.2% of all cache accesses. Because they are bound to specific conversations, their lifetimes are

¹While recent research has demonstrated long-running agents [33, 36, 69], most of the agents in production solve simpler tasks and run shorter.

Table 5: Summary of the characteristics that separate the prefix-cache workload from the web and block workloads. Each characteristic carries the subsection that measures it.

Characteristic	Prefix cache	Web	Block
Lifetime (§4.1)	Minutes, bounded by the session	Days with a persistent hot core	Days without hot core
Reuse interval (§4.2)	Short, low variance	Long, high variance	Long, paced by periodic jobs
Frequency (§4.3)	Wide spread; tracks session progress	Zipfian; a few get most accesses	Few accesses; scans dominate
Miss penalty (§4.4)	Grows with block depth	One uniform fetch	One uniform fetch
Session footprint (§4.5)	Heavy-tailed per session	No session structure	No session structure

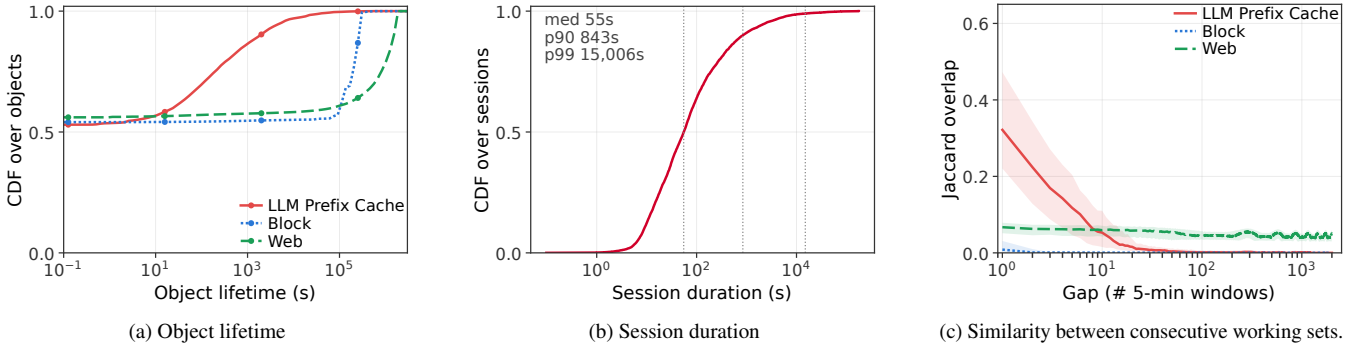


Figure 3: (a) Prefix-cache blocks are short-lived, with lifetimes up to 100× shorter than in traditional caches. (b) Sessions are short: P90 is 14 minutes, while the median is under 1 minute. (c) Working set similarity is high at very short intervals *due to short reuse* but rapidly falls to zero *due to short lifetime*.

strictly confined to their originating session (median 219 s).

Single-turn request prompts. Prompts from single-turn requests form the largest fraction of unique blocks (57.8%) but contribute only 11.4% of total accesses. The vast majority exhibit virtually no reuse: 84.9% of these blocks are accessed exactly once. The remaining 15.1% of single-turn blocks correspond to shared prefixes across different independent requests (such as common task templates).

This session-bounding also fundamentally changes working set churn. Figure 3c shows the Jaccard overlap between two 5-minute access windows over time. Web workloads maintain a persistent ~5% globally popular core, while the block trace exhibits near-zero overlap. The prefix-cache working set, however, completely lacks a persistent core: while adjacent windows share over 30% of their objects, this overlap decays to near zero beyond a two-hour gap as sessions conclude.

This rapid churn and strict session-bounding have profound implications for cache sizing and eviction. The working set size is straightforward to estimate—roughly the block arrival rate multiplied by the expected session duration. As shown in Figure 2, the 96 GiB hit ratio is very high despite caching capacity under 0.2% of unique blocks. More critically, dedicating capacity to retain older blocks yields no benefit, since overlap vanishes after two hours.

Finding & Implication: Prefix-cache working sets churn rapidly: blocks have short lifetimes and lack the persistent hot set common in traditional cache workloads. As a result, most workloads do not need a large cache size (relative to unique data).

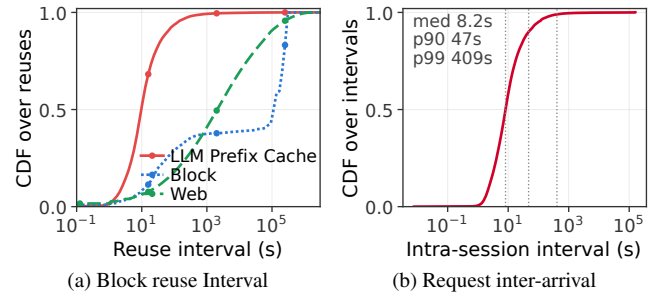
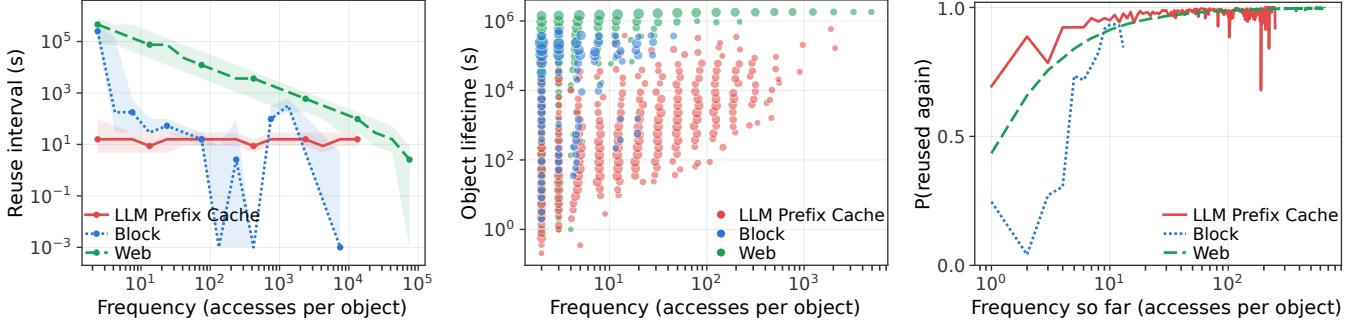


Figure 4: (a) Prefix-cache reuse intervals are short with low variance. (b) Request inter-interval time within a session is short.

4.2 Recency Provides a Key Signal

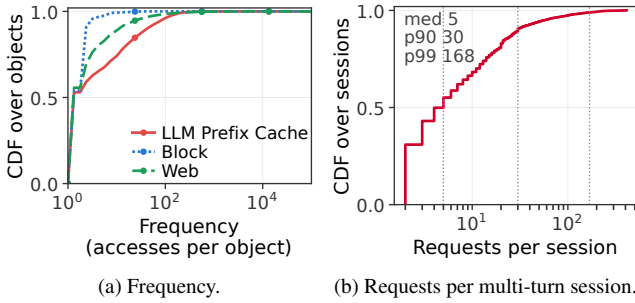
Prefix-cache reuse intervals are remarkably short and concentrated, contrasting sharply with the long, high-variance intervals of traditional workloads, as shown in Figure 4a. Web interarrival times reflect independent users, producing smooth curves with a wide interquartile range (P25–P75). Block traces mix diverse behaviors like periodic scans, causing sharp modal spikes. Prefix caching, however, is dominated by rapid intra-session interactions. Figure 4b highlights this extreme recency. Driven by quick tool executions and conversational turns, the median intra-session interval is a mere 8.2 s, and 99.7% of all gaps fall under a 22-minute reuse window.

While the median reuse interval is short across all block categories (10.0 s for system prompts, 11.2 s for multi-turn history), they do exhibit a heavy tail, with the 99th percentile



(a) Reuse interval vs. frequency. Lines show the median and bands the P25–P75 range. (b) Object lifetime vs. frequency. Bubble area indicates the number of objects in each bin. (c) Probability of further reuse vs. current frequency.

Figure 5: Frequency is not a good indicator for LLM prefix cache. (a) Reuse intervals are largely independent of block popularity in the LLM prefix cache. (b) Block lifetime increases with access frequency and shows a wide spectrum. (c) Frequency is a poor predictor of future reuse.



(a) Frequency. (b) Requests per multi-turn session.

Figure 6: (a) Prefix-cache blocks are reused more frequently than in web and storage block caches. (b) Requests per session show a wide spectrum.

reaching 1,000 s and 562 s respectively. This tail is caused by extended human interaction and prolonged tool executions, which can delay the next request up to several days.

Despite this heavy tail, because the overwhelming majority of reuses are concentrated in extremely short intervals, recency remains the dominant signal for whether a block should be retained. When a block goes idle for a long period, it should be aggressively evicted regardless of its past popularity. This is because a long-idle block likely belongs to a working set that has already churned away, meaning it will probably not be reused. Furthermore, even if the block does fall into the heavy tail and is eventually reused, the capacity cost of pinning it in memory throughout a prolonged idle duration far outweighs the benefit of a single cache hit. This explains why simple recency-based eviction algorithms perform so well in our initial study of existing algorithms (§3.2).

Finding & Implication: Eviction should prioritize recency: reuse is concentrated within short intervals, long-idle blocks should be reclaimed aggressively, as they are unlikely to be reused and increasingly costly to retain.

4.3 Frequency Tracks Session Progress

Prefix-cache objects exhibit a vastly wider spread of frequency than traditional workloads. As demonstrated in Figure 6a, while web frequencies plateau early due to Zipfian popularity and block objects see minimal reuse, prefix-cache frequency fundamentally tracks session progress. As sessions advance, their history blocks are repeatedly re-read. Because a block’s reuse count depends on both the overall session length and its insertion turn, reuses span from a handful to hundreds of times. Figure 6b shows that the number of turns per session is exceptionally large and heavy-tailed, reaching up to 412 requests. This massive variation in session lengths drives extreme frequency variance across the cache, with system prompts and multi-turn history blocks reaching median accesses of 15 and 12 respectively. Such prolonged interactions also cause immense context growth, making partial eviction highly valuable for preserving the active context.

Unlike traditional workloads, accumulated frequency in prefix caching provides almost no signal about the timing of the next request. As shown in Figure 5a, web and block traces exhibit a downward trend where higher popularity translates to shorter reuse intervals due to concurrent user requests. In contrast, the prefix-cache reuse interval remains completely flat across all frequency bins, demonstrating that accesses arrive at the stable, sequential pace of individual sessions. Consequently, prefix-cache object lifetime increases as frequency grows, as demonstrated in Figure 5b. Web and block lifetimes, however, remain decoupled from frequency, since objects drawn from a global pool stay active indefinitely with little reuse. Because frequency fails to predict shorter reuse intervals, it proves highly ineffective for ranking eviction candidates. This fundamental disconnect explains why frequency-heavy algorithms suffer such poor performance in our initial algorithm comparison (§3.2).

However, early frequency is an excellent predictor for *whether* a block will be reused at all. As shown in Figure 5c, prefix caching exhibits a uniquely sharp increase in future

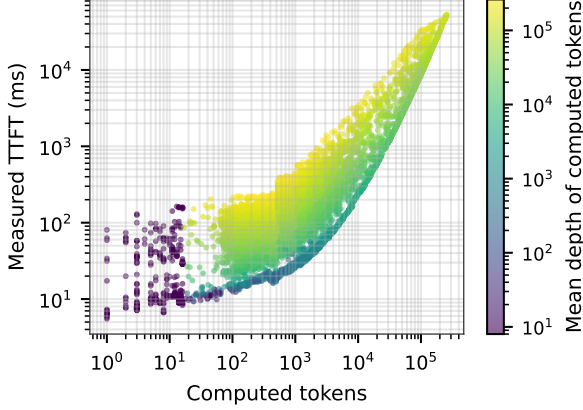


Figure 7: Measured TTFT against the number of tokens computed per request, over 10,000 FreeInference Trace requests replayed on vLLM [29] with Qwen3-Coder-30B [59] under LRU at a 48 GiB cache on a single NVIDIA H200. Color indicates the mean depth (position in the prompt) of the computed tokens. Similar token counts can incur substantially different TTFT.

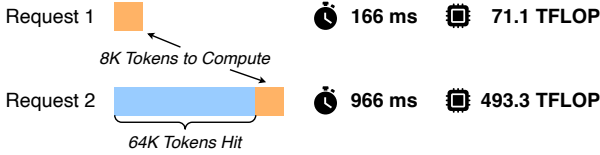


Figure 8: Depth-dependent recomputation cost on Qwen3-Coder-30B [59], measured on NVIDIA H200. Both requests compute 8K tokens, but the second must attend to an additional 64K cached tokens, increasing computation and TTFT.

reuse probability during the first few hits. Securing just a second hit jumps this probability from 69.4% to 88.7%, cleanly separating single-turn requests from established multi-turn sessions. For workloads flooded with one-hit wonders, this implicit classification makes early frequency a helpful admission filter to discard transient prompts. We explore this conditional benefit further in §5.1, demonstrating how quick demotion specifically aids the Qwen To-B trace [64].

Finding & Implication: Because session accesses arrive at a steady pace, frequency fails to predict reuse intervals and should not be used as an eviction ranking metric. However, for workloads dominated by one-hit wonders, early frequency can serve as a helpful admission filter.

4.4 Position-Dependent Miss Costs

Hit ratio alone does not determine the benefit of prefix caching. Figure 7 plots measured TTFT against the number of tokens computed per request. Even among requests computing a comparable number of tokens, TTFT varies substantially. The color of each point shows that requests computing tokens deeper in the prompt generally incur higher TTFT at a similar token count. Thus, the absolute number of tokens left to

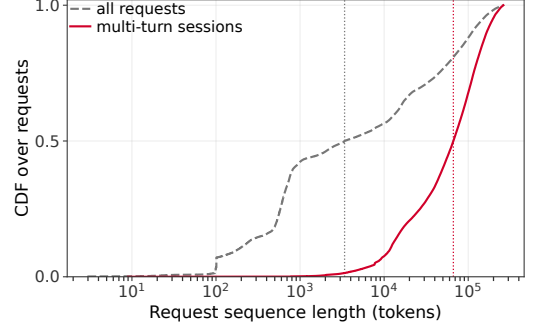


Figure 9: Distribution of request sequence lengths in tokens, over all requests and over the requests that belong to multi-turn sessions. Long requests both occupy substantial HBM and create deep blocks with high recomputation costs. Dotted lines mark the two medians, 3,386 and 66,605 tokens.

compute is insufficient to predict TTFT.

Figure 8 illustrates why depth matters. Both requests compute 8K tokens, but one starts from scratch while the other follows a cached 64K-token prefix. Each uncached token must attend to all preceding tokens in full-attention layers, including those cached tokens. Hence, the second request performs more attention work despite computing the same number of tokens. For a fixed-size block, this attention work grows with its depth, making misses on deeper blocks more expensive.

These cost differences matter because request lengths span a wide range (Figure 9), reaching 111K tokens at the 90th percentile. Such long contexts contain blocks with vastly different recomputation costs, despite occupying identical cache space and contributing equally to hit ratio.

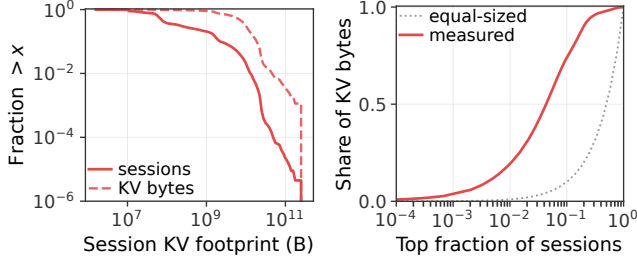
Eviction algorithms should therefore account for the computation saved by each hit. When blocks have comparable expected reuse, retaining deeper, more expensive blocks and evicting cheaper, shallow blocks can reduce recomputation cost for the same number of hits. The separation of cache residency from prefix matching (§2.3) makes this possible by allowing shallow blocks to be recomputed while deeper cached blocks are reused. We will evaluate compute-aware eviction and its implementation tradeoffs in §5.2.

Finding & Implication: Hit ratio alone does not determine TTFT because deeper blocks save more computation. Eviction should thus account for this depth-dependent cost.

4.5 Heavy-Tailed Session Footprints

Session KV footprints span several orders of magnitude. Figure 10a shows the median session accumulates just 45 blocks (0.07 GiB), whereas the 99th percentile reaches 8.3K blocks (12.2 GiB) and the maximum hits 234 GiB. These massive footprints emerge as context accumulates across turns (§4.3) and scales with the long requests shown in Figure 9, allowing a single session to easily overwhelm the HBM cache.

Consequently, KV memory concentrates intensely in the



(a) Footprint tail (CCDF), by sessions and KV bytes. (b) Concentration of KV bytes across sessions.

Figure 10: Per-session KV footprint of the prefix-cache trace.

largest sessions. The Lorenz curve in Figure 10b reveals the top 10% of sessions hoard 76.2% of all KV bytes, with the top 1% alone consuming 20.3%. By partially trimming these heavy sessions, cache managers can reclaim substantial capacity without destroying the entire reusable context.

This extreme skew forces eviction granularity to adapt to capacity constraints. When cache space is tight, treating a massive session as an indivisible unit forces a destructive all-or-nothing choice, while fine-grained eviction gracefully avoids this by reclaiming selected blocks. Conversely, in an expansive secondary pool, session-level management becomes highly attractive for drastically reducing metadata overhead. We quantify this tradeoff in §5.3.

Finding & Implication: Session KV footprints are heavy-tailed. Constrained caches require partial-session eviction, while larger pools may instead trade fine-grained eviction for lower metadata overhead.

4.6 Findings Generalize Across Workloads

To test whether our findings generalize beyond FreeInference Trace, we extend the algorithm comparison from §3.2 to two production Qwen Baillian traces [64] and AgentX [53]. We evaluate a six-algorithm subset at four cache capacities in Figure 11, selecting canonical algorithms for each category.

AgentX replays 393 Claude Code sessions from the Semi-Analysis corpus [53]. Because the corpus lacks absolute timestamps, we artificially schedule sessions to maintain a steady concurrency of 16, while preserving the exact timing between requests within each session. Because its hashes are session-scoped, cross-session sharing is modeled as zero, making the AgentX hit ratios a conservative lower bound.

The results confirm the core role of recency (§4.2). At each capacity, the best online algorithm is usually recency-based, and by 1 TiB, LRU is within 0.4 points of the best online algorithm on every trace. Conversely, frequency-based W-TinyLFU consistently underperforms (§4.3).

These three traces represent distinct serving workloads. Across them, the recency conclusion is stable, but the capacity at which LRU catches up is workload-dependent. The Qwen traces contain less reusable content than FreeInference Trace:

compulsory misses account for 34–54% of accesses, compared with 6.0% on FreeInference Trace. Conversely, AgentX contains even more reusable content, with only 3.75% compulsory misses. Accordingly, the offline optimum at 1 TiB is 0.534 (To-B), 0.664 (Coder), and 0.962 (AgentX). At 24 GiB, LRU does not lead on any trace; on Qwen To-B, for example, S3-FIFO and ARC reach 0.491 versus LRU’s 0.403. LRU catches up only at 60–462 GiB, compared with 37 GiB on FreeInference Trace. Thus, some workloads require substantially more cache capacity before recency alone is sufficient. This motivates the conditional use of quick demotion in §5.1. Appendix A presents line plots across the full capacity range for the four Qwen traces, AgentX, and the two traces evaluated in §3, each with the full algorithm set; Appendix B analyzes sensitivity to session concurrency at fixed capacity.

5 What Designs Can Be Helpful?

The empirical policy behaviors observed in §3 and the design implications uncovered in §4 demonstrate that while a standard recency-based policy like LRU provides a remarkably robust baseline, no single caching algorithm universally wins across all deployment scenarios. Because optimal performance is intimately tied to the interaction between workload characteristics and cache capacity, we do not propose a monolithic “best” policy. Instead, we explore a set of modular algorithmic techniques that can be selectively deployed to address the specific bottlenecks of prefix caching.

LRU as a bedrock. Because recency is the primary predictor of future reuse (§4.2), Least Recently Used (LRU) serves as the foundational baseline for our proposed designs. As shown in §3, plain LRU is remarkably robust, reliably converging to the optimal offline ceiling on every trace we measured while requiring the absolute minimum metadata overhead. Consequently, rather than reinventing the core eviction logic, we use LRU as the bedrock and introduce several targeted additions designed to handle specific edge cases and constrained environments where pure recency falls short.

5.1 Quick Demotion

When capacity is severely constrained and the workload is flooded with single-turn requests, recency alone can struggle to protect the active working set. In environments dominated by one-hit wonders, *quick demotion* strategies like S3-FIFO [72] can be highly effective by rapidly filtering out blocks that will never be reused (§4.3). Figure 11 (Qwen To-B panel) shows the value of this filtering: at 24 GiB, S3-FIFO achieves a 0.491 hit ratio compared to LRU’s 0.403.

However, one-hit wonders typically belong to short, single-turn prompts, meaning they occupy a small cache footprint compared to multi-turn sessions. Therefore, quick demotion provides substantial benefits only under specific extremes: when one-hit prompts account for such an overwhelming

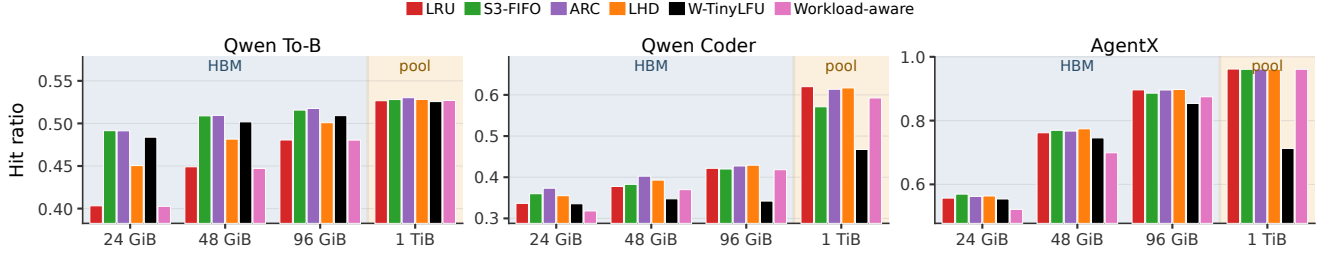


Figure 11: Block hit ratio of six eviction algorithms at the four capacities of Table 2, on two of the four Qwen Baillian traces [64] and on AgentX, a replay of 393 Claude Code sessions [53].

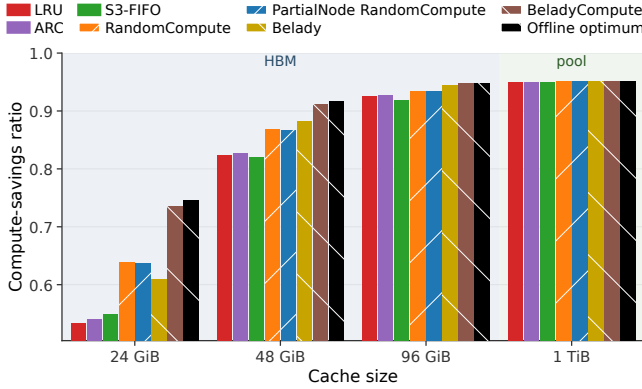


Figure 12: Compute-savings ratio on FreeInference Trace under the Qwen3-Coder-30B [59] cost model. BeladyCompute tightly approximates the ILP optimum, whereas standard Belady falls short. Compute-aware eviction substantially improves compute savings, and partial-node eviction preserves these gains without degradation.

volume of traffic that their cumulative footprint genuinely threatens to flood the cache.

Takeaway. Quick demotion strategies like S3-FIFO are effective, but should only be enabled when one-hit prompts account for an extremely high volume of traffic.

5.2 Compute-awareness

Because a cache miss in LLM serving triggers recomputation, eviction decisions should ideally optimize for compute savings rather than pure hit counts, especially when the cost of recomputation varies widely across blocks. To evaluate this, we introduce the *compute-savings ratio*, a metric that weights each cache hit by the computational cost avoided. Under this framework, the traditional hit ratio simply represents a constant cost model that regards all blocks as having the exact same cost. To capture real-world variance, we model compute intensity based on the FLOPs required for each block on Qwen3 Coder 30B. Figure 12 illustrates the compute-savings ratio of various policies under this cost model. We additionally evaluate an extreme linear cost model where full attention strictly dominates, finding that all conclusions below hold with slightly wider margins (Appendix C).

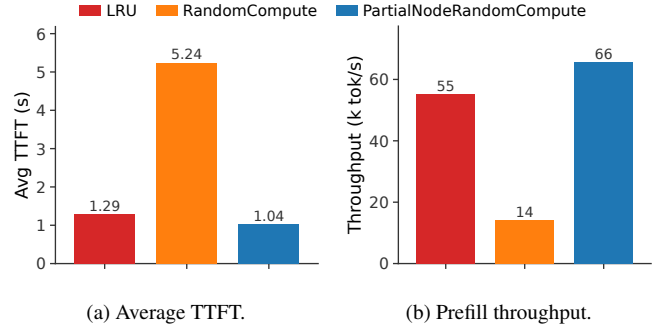


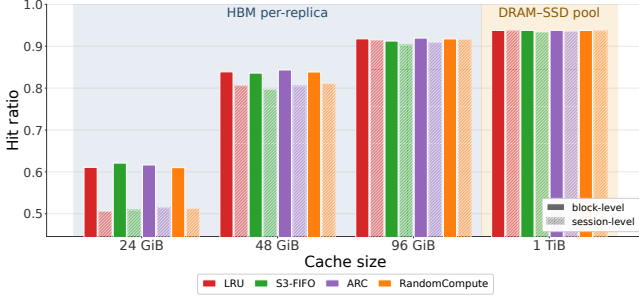
Figure 13: End-to-end performance on H200 with 48 GiB cache replaying 10,000 requests from FreeInference Trace. TTFT (a) and throughput (b) were measured in separate runs. Partial-node compute-aware eviction yields substantial TTFT and throughput gains over LRU, whereas block-based eviction incurs severe degradation due to fragmentation.

To establish a theoretical optimum for this metric, we frame the caching process as an Integer Linear Programming (ILP) problem, using reuse intervals as variables and capacity as a constraint. Because solving the ILP is computationally expensive, we also introduce *BeladyCompute*: an efficient offline algorithm inspired by BeladySize [71]. For a given block i , BeladyCompute evaluates an eviction score:

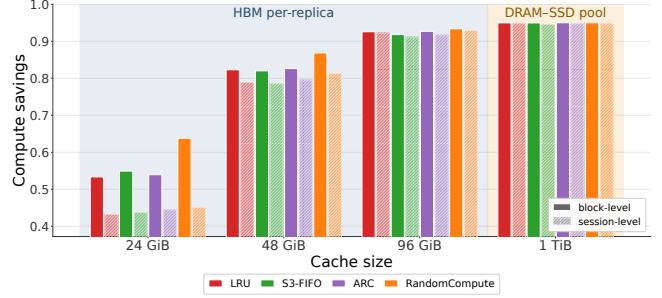
$$\text{Score}(i) = \text{ComputeIntensity}(i) \times \text{TimeUntilNextAccess}(i)$$

and evicts the block with the highest score. As shown in Figure 12, BeladyCompute tracks the offline ILP optimum exceptionally well across all constrained cache sizes—falling just 1.03 points below the optimum at 24 GiB, 0.41 points at 48 GiB, and a 0.08 points at 96 GiB. This proves it is a robust and efficient approximation of the absolute theoretical limit.

These offline oracles offer an easy diagnostic for whether a workload deserves compute-awareness. The gap in compute-savings ratio between the hit-optimal Belady and cost-aware BeladyCompute bounds the potential gain. On FreeInference Trace, this gap is massive (+12.6 points at 24 GiB), justifying the design effort. Conversely, on the other five traces, the gap is negligible (≤ 1.1 points), meaning standard hit-optimal evictions is good enough. Thus, running both oracles on a trace sample can decisively evaluate the necessity of compute-awareness before designing any online policy. Because this



(a) Hit ratio at four capacities.



(b) Compute savings at four capacities.

Figure 14: Block-level (solid) vs. session-level (hatched) eviction on the prefix-cache trace. While session-level eviction matches block-level performance at large capacities, it degrades under tight constraints.

cost profile depends primarily on the model and the proportion of long sequences, the diagnostic only needs to be repeated when these factors change.

For practical online environments, we design *RandomCompute*, a compute-aware algorithm that targets cheap, shallow, and long-idle blocks for eviction. It achieves this by randomly sampling a subset of blocks in the cache and evicting the one with the lowest score, calculated for each sampled block i as:

$$\text{Score}(i) = \frac{\text{ComputeIntensity}(i)}{\text{TimeSinceLastAccess}(i)}$$

In long-context scenarios with high variance in block computation costs, this strategy yields massive efficiency gains. At a constrained 24 GiB cache under this cost model, *RandomCompute* achieves a 0.638 compute-savings ratio, outperforming LRU by 10.4 points and even surpassing the standard hit-optimal Belady oracle [6].

However, as a sample-based algorithm, *RandomCompute* introduces a severe problem: it heavily fragments the prefix, leaving behind numerous “holes”² that creates massive overhead of the attention matrix computation [61]. *RandomCompute* leaves an average of 9.21 holes per request, with a long tail reaching hundreds of holes. Appendix D reports the full distributions. To mitigate this fragmentation, we introduce *partial-node* eviction, which drops contiguous chunks of blocks from the beginning of a node rather than scattered individual blocks. Preferably, this chunk size should align with the system’s contiguous batching token budget (if applicable). This alignment ensures that hole-filling perfectly utilizes the budget while minimizing any reduction in compute savings. Furthermore, because there are no actual computational dependencies between missing segments, the prefill for different holes can be batched and executed entirely in parallel. By packing the same recomputation volume into fewer and longer holes, *PartialNode RandomCompute* slashes the average hole count by 8.7×. This optimization effectively eliminates the parallelization bottleneck while sacrificing negligible compute savings at 24 GiB, making it a highly practical design.

²A hole is a contiguous run of blocks a request has to recompute.

We validate these gains on a real vLLM engine serving Qwen3-Coder-30B [59] with a 48 GiB cache (Figure 13). Against LRU, our partial-node compute-aware design cuts average Time-To-First-Token (TTFT) by 19.9% (1.04 s vs. 1.29 s) and raises prefill throughput by 18.8% (65.6K vs. 55.2K tokens/s). Fragmentation penalties render the original block-granular *RandomCompute* unusable: it incurs 4.0× higher TTFT and 3.9× lower throughput than LRU. Mechanistically, our policy trades cheap shallow hits to protect expensive deep ones: while LRU retains a slightly better median TTFT (112 ms vs. 171 ms), our design decisively slashes the 99th-percentile tail from 26.0 s down to 16.8 s.

Takeaway. When sequence lengths are long and recomputation costs vary significantly, compute-aware eviction delivers massive efficiency gains. However, practical online implementations must evict blocks in contiguous chunks (e.g., *partial-node* eviction) to avoid severe memory fragmentation that would otherwise cripple parallel attention computation.

5.3 Block-Level vs. Session-Level Eviction

When managing secondary KV cache pools [51, 68], maintaining metadata for millions of individual blocks incurs a significant overhead. For example, tracking a 12 TiB pool requires indexing 8.4 million 16-token blocks, so systems like Mooncake [51] and LMCACHE [40] use 16–32× coarser chunks (256–512 tokens). *Session-level* eviction drastically reduces this overhead by evicting entire sessions at once, but sacrifices the granular efficiency of partial eviction.

Figure 14 illustrates this fundamental tradeoff. At highly constrained cache sizes (e.g., 24 GiB), session-level eviction is disastrous: LRU loses 10.5 points of hit ratio, and the compute-aware *RandomCompute* policy loses a staggering 18.7 points of compute savings. When a giant session monopolizes the cache, failing to partially trim it forces many valuable sessions out. The penalty reduces but stays material at the 48 GiB, where LRU still gives up 3.3 points of hit ratio and *RandomCompute* 5.5 points of compute savings. Since blocks are already the native management granularity in HBM, accept-

ing this penalty offers no system benefit. This severe penalty falls off quickly as capacity grows. In the large pool regime, the performance gap between session-level and block-level eviction becomes virtually unmeasurable. The only permanent drawback to session granularity is cross-session sharing: evicting an entire session inadvertently purges shared system prompts that other active sessions still need, though the vast capacity of the pool largely masks this effect.

Takeaway. Eviction granularity needs to match the memory tier. In the HBM regime, systems should strictly employ block-level eviction to maximize performance, which naturally align with native block management as well. Conversely, given the performance penalty vanishes in the large pool regime, designers can freely choose between block-level and session-level eviction based on metadata organization.

5.4 Summary

In summary, designing a cache for prefix-caching workloads requires an approach tailored to specific capacity constraints and workload characteristics. If the workload is flooded with an extremely high volume of short, single-turn prompts that threatens to overwhelm capacity, systems should enable quick demotion (e.g., S3-FIFO) to filter them out early. If the system serves models with heavy attention complexity and long sequences, designers should switch to a compute-aware eviction algorithm (paired with partial-node granularity) to prioritize retaining expensive blocks without causing fragmentation. When combined with quick demotion, this compute-awareness should only be applied to the main cache to prevent the admission queue from prematurely evicting shallow blocks. Finally, if managing a constrained HBM cache, strictly employ block-level eviction to maximize performance; conversely, if managing a massive secondary KV cache pool, designers can flexibly choose either block-level or session-level eviction, since the latter drastically reduces metadata overhead without incurring a performance penalty.

6 Discussion

6.1 Other Attention Designs

While blocks are uniformly sized, the underlying attention mechanism dictates a highly variable recomputation cost. Our three cost models ablate this dynamic, spanning from a uniform-cost baseline (§3.2) to the Qwen3-Coder-30B cost (§5.2) and an idealized linear model (Appendix C). Alternative attention designs dictate where a given deployment falls within this spectrum. Sliding-window attention [25] caps cost growth with prefix depth and moves the objective closer to constant cost, while in hybrid attention models [37], the benefit scales with the share of full-attention layers. GQA [2], MQA [54], MLA [17], cross-layer KV sharing [9], and KV quantization [42] can also change recomputation cost. These

designs require recalibrating the cost model, while the reuse patterns in §4 still apply.

6.2 Interaction with Load Balancing

While routing load balancers often use cache affinity to raise local reuse, they can create uneven cache pressure across replicas due to the heavy-tailed session footprints we identified (§4.5) [24, 65]. Regardless of how a router distributes these requests across the cluster, our findings and policy designs remain fully applicable. Once a request lands on a specific replica or shared memory pool, the underlying access patterns—session-bounded lifetimes (§4.1), concentrated recency (§4.2), and varying compute costs (§4.4)—still strictly dictate the cache dynamics. Consequently, our recency-based eviction insights, quick demotion, and compute-aware optimizations hold true as the foundational eviction layer beneath any load balancing strategy.

6.3 SSD Offloading

Given the extremely short, session-bounded lifetimes of prefix-cache blocks (§4.1), offloading evicted blocks from HBM to an SSD tier [40] is generally unnecessary for most workloads, as the working set churns away before the SSD capacity provides any benefit. However, in specific cases where workloads exhibit long-tail reuse or extreme capacity demands, SSD offloading may still be deployed. In these scenarios, a practical SSD tier must employ strict selective admission to preserve limited flash endurance [52] while capturing the narrow window of actual reuse [19, 44].

7 Related Work

Prefix caching and storage. vLLM [29] and SGLang [76] enable prefix reuse via paged and radix-tree KV caches. Mooncake, Strata, and PTStore [43, 51, 68] extend KV storage across GPU memory, host memory, SSDs, and remote nodes. Cache-dAttention and IMPRESS [14, 23] manage retention across these tiers. We study the replacement policy within such mechanisms and how it should change with the memory tier.

Prefix-cache eviction. Recent policies predict session continuation, partition workloads, or rank blocks using recency, frequency, and semantic features [20, 34, 46, 56, 64, 70]. Marconi [47] models compute saved per byte for hybrid attention models. We instead compare classical, learned, and prefix-specific policies over two production traces and a wide capacity range, then explain their behavior through session-bounded reuse, frequency (accesses per object), non-uniform miss cost, and session footprint. These properties motivate our compute-aware and tier-dependent designs.

Adjacent cache decisions. CacheWise [60] and KVFlow [48] use tool or workflow metadata unavailable in the access stream to predict reuse. Continuum [31], InferCept [1], and

EvicPress [21] decide whether to preserve or move one request’s context, while cache-aware routers [24, 65] decide where a request runs. These signals and decisions complement eviction, which determines what remains available at each replica.

Workload characterization. Production studies characterize prompt composition, sessions, and observed cache reuse [3, 38, 64, 79]. However, existing public traces lack the combination of agentic traffic, accurate multi-day timestamps, and fine-grained block hashes necessary for robust policy simulation. By introducing traces with these properties, our controlled trace replay enables comparing eviction policies and their offline optimum on the same request stream.

Approximate KV-cache management. H2O [75], Scissorhands [41], StreamingLLM [67], and SnapKV [35] remove KV entries within one active context; Quest [58] and InfiniGen [30] skip or prefetch them. These methods may trade output quality for per-request memory, whereas our cross-request blocks are exactly reconstructible and a miss costs prefill compute. Our evaluated policies draw from classical recency, frequency, admission, adaptive, analytic, and learned replacement [5, 26, 45, 57, 62, 72, 77], with Belady’s algorithm as the offline optimum [6].

8 Conclusion

As LLM serving scales, prefix caching has become essential to bypass expensive prefill recomputations, yet we find that advanced eviction algorithms built for traditional storage yield surprisingly little improvement over simple LRU. Our systematic trace analysis uncovers the root cause: prefix reuse follows the steady pacing of active sessions, making recency very crucial and rendering frequency-based signals ineffective. We also identify unique challenges in prefix caching: highly non-uniform miss costs that scale with depth and heavy-tailed session footprints. To optimize beyond simple hit ratios, systems must retain a recency foundation but augment it conditionally: deploying quick demotion to filter one-hit wonders, introducing compute-aware eviction to prioritize expensive blocks, and adapting eviction granularity to the capacity tier. Together, these targeted optimizations provide a concrete foundation for scalable prefix caching.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235, pages 81–95, 2024.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [3] Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. State of ai: An empirical 100 trillion token study with openrouter. *arXiv preprint arXiv:2601.10088*, 2026.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137, 2024.
- [5] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. LHD: Improving cache hit rate by maximizing hit density. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 389–403, 2018.
- [6] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.
- [7] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosf, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. The CacheLib caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [8] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. AdaptSize: Orchestrating the hot object memory cache in a content delivery network. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 483–498, 2017.
- [9] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing Transformer key-value cache size with cross-layer attention, 2024.
- [10] Lee Breslau, Pei Cao, Li Fan, Graham Phillips, and Scott Shenker. Web caching and Zipf-like distributions: Evidence and implications. In *Proceedings of IEEE INFOCOM ’99: Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies*, volume 1, pages 126–134, 1999.
- [11] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda

- Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [12] cacheMon. The cacheMon cache trace collection. https://github.com/cacheMon/cache_dataset, 2024. Accessed September 2026.
- [13] Character.AI. Optimizing AI inference at Character.AI, 2024.
- [14] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. IMPRESS: An importance-informed multi-tier prefix KV storage system for large language model inference. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 187–202, 2025.
- [15] Ludmila Cherkasova. Improving WWW proxies performance with Greedy-Dual-Size-Frequency caching policy. Technical Report HPL-98-69R1, Hewlett-Packard Laboratories, November 1998.
- [16] Chutes. <https://chutes.ai/>.
- [17] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [18] Gil Einziger, Roy Friedman, and Ben Manes. TinyLFU: A highly efficient cache admission policy. *ACM Transactions on Storage*, 13(4), 2017.
- [19] Assaf Eisenman, Asaf Cidon, Evgenya Pergament, Or Haimovich, Ryan Stutsman, Mohammad Alizadeh, and Sachin Katti. Flashfield: a hybrid key-value cache that controls flash write amplification. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019.
- [20] Shaoke Fang, Ziang Li, Wenfei Wu, Jiatong Ji, Qingsong Liu, and Ruizhi Pu. Not all tokens are worth caching: Learning semantic-aware eviction for LLM prefix caches, 2026.
- [21] Shaoting Feng, Yuhang Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. EviPress: Joint KV-cache compression and eviction for efficient LLM serving, 2025.
- [22] FreeInference. <https://freeinference.org/>.
- [23] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (ATC)*, pages 111–126, 2024.
- [24] Philip Hayes. Accelerate multi-turn LLM workloads on OpenShift AI with llm-d intelligent routing, 2026.
- [25] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, et al. Mistral 7B, 2023.
- [26] Song Jiang and Xiaodong Zhang. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In *Proceedings of the 2002 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, 2002.
- [27] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [28] Swaroop Kavalanekar, Bruce Worthington, Qi Zhang, and Vishal Sharda. Characterization of storage workload traces from production Windows servers. In *2008 IEEE International Symposium on Workload Characterization (IISWC)*, pages 119–128, 2008.
- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [30] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 155–172, 2024.
- [31] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Hanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live, 2025.
- [32] Jinhong Li, Qiuping Wang, Patrick P. C. Lee, and Chao Shi. An in-depth analysis of cloud block storage workloads in large-scale production. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pages 37–47, 2020.
- [33] Keyu Li, Junhao Shi, Yang Xiao, Mohan Jiang, Jie Sun, Yunze Wu, Dayuan Fu, Shijie Xia, Xiaojie Cai, Tianze Xu, Weiye Si, Wenjie Li, Dequan Wang, and Pengfei Liu. AgencyBench: Benchmarking the frontiers of autonomous agents in 1M-Token real-world contexts, 2026.

- [34] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. HotPrefix: Hotness-aware KV cache scheduling for efficient prefix sharing in LLM inference systems. *Proceedings of the ACM on Management of Data*, 3(4), 2025. Article 250 (SIGMOD).
- [35] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pages 22947–22970, 2024.
- [36] Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, Lichao Sun, Haotao Mi, and Leowei Liang. Long-Horizon-Terminal-Bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading, 2026.
- [37] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, et al. Jamba: A hybrid Transformer-Mamba language model, 2024.
- [38] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot at production scale, 2026.
- [39] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [40] Yuhan Liu, Jiayi Yao, Yihua Cheng, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference, 2025.
- [41] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 52342–52364, 2023.
- [42] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [43] Meghana Madhyastha, Robert Underwood, Randal Burns, and Bogdan Nicolae. PTStore (prefix tensor store): Distributed prefix caching and replication for high throughput inference serving, 2026.
- [44] Sara McAllister, Benjamin Berg, Julian Tutuncu-Macias, Juncheng Yang, Sathya Gunasekar, Jimmy Lu, Daniel S. Berger, Nathan Beckmann, and Mor Harchol-Balter. Kangaroo: Caching billions of tiny objects on flash. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021.
- [45] Nimrod Megiddo and Dharmendra S. Modha. ARC: A self-tuning, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*, 2003.
- [46] Bei Ouyang, Yifan Qiao, and Jiarong Xing. UniCache: Unifying prefix cache eviction for heterogeneous LLM serving workloads. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 10(2), 2026. Article 54.
- [47] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [48] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows, 2025.
- [49] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [50] Purav. Why Manus agents don’t break the cache and yours probably do, 2025.
- [51] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*, pages 155–170, 2025.
- [52] Bianca Schroeder, Raghav Lagisetty, and Arif Merchant. Flash reliability in production: The expected and the unexpected. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, 2016.

- [53] SemiAnalysis. AgentX methodology and datasets (v1.0). <https://inference.semianalysis.com/agentx>, 2026. Trace corpus at <https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126>. Accessed September 2026.
- [54] Noam Shazeer. Fast Transformer decoding: One write-head is all you need, 2019.
- [55] Sheng Shen, Le Hou, Yanqi Zhou, Nan Du, Shayne Longpre, Jason Wei, Hyung Won Chung, Barret Zoph, William Fedus, Xinyun Chen, et al. Mixture-of-experts meets instruction tuning: A winning combination for large language models. *arXiv preprint arXiv:2305.14705*, 2023.
- [56] Chunan Shi, Yilei Chen, Yilin Chen, Xupeng Miao, and Bin Cui. Multi-segment attention: Enabling efficient KV-Cache management for faster large language model serving, 2026.
- [57] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020.
- [58] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 47901–47911, 2024.
- [59] Qwen Team. Qwen3 technical report, 2025.
- [60] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. CacheWise: Understanding workloads and optimizing KVCache management for efficiently serving LLM coding agents, 2026.
- [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [62] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX Association, 2018.
- [63] Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient MRC construction with SHARDS. In *13th USENIX Conference on File and Storage Technologies (FAST)*, pages 95–110, 2015.
- [64] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, July 2025.
- [65] Jiahao Wang, Kaizhan Lin, Kaixi Zhang, Jinbo Han, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Rong Chen, and Haibo Chen. SMetric: Rethink LLM scheduling for serving agents with balanced session-centric scheduling, 2026.
- [66] Haocheng Xia, William Nixon, Bintang Dwi Marthen, Pranav Bhandari, and Juncheng Yang. Learning-augmented heuristics: Simple yet smart, robust and interpretable cache eviction. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 2241–2259, 2026.
- [67] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [68] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving. *arXiv preprint arXiv:2508.18572*, 2025.
- [69] Zhangchen Xu, Junda Chen, Yue Huang, Dongfu Jiang, Jiefeng Chen, Hang Hua, Zijian Wu, Zheyuan Liu, Zexue He, Lichi Li, Shizhe Diao, Jiaxin Pei, Jinsung Yoon, Hao Zhang, Mengdi Wang, Radha Poovendran, Misha Sra, Alex Pentland, and Zichen Chen. AutoLab: Can frontier models solve long-horizon auto research and engineering tasks?, 2026.
- [70] Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [71] Juncheng Yang. libCacheSim: A high-performance library for building cache simulators. <https://github.com/1alalla/libCacheSim>, 2024. Accessed September 2026.
- [72] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. Fifo queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 130–149, 2023.
- [73] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React:

Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.

- [74] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024.
- [75] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 34661–34710, 2023.
- [76] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [77] Wenbin Zhou, Zhixiong Niu, Yongqiang Xiong, Juan Fang, and Qian Wang. 3L-Cache: Low overhead and precise learning-based eviction policy for caches. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, 2025.
- [78] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35:7103–7114, 2022.
- [79] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Trace-Lab: Characterizing coding agent workloads for LLM serving, 2026.

A Capacity Sweeps

Sections 4.6 and 3.2 compare policies at four cache capacities on six traces. Figures 15 and 16 extend this comparison to 65 capacities across both cache-size regimes. Both sweeps include every policy in Figure 2 except LRB. We omit LRB because it is prohibitively slow to sweep and never outperforms LeCaR at the four sampled capacities on either production trace.

The denser sweeps expose two behaviors hidden by the four sampled capacities. First, the gains from quick demotion can reverse as capacity grows. On Qwen To-C, for example, LIRS moves from 4.2 points above LRU to 10.1 points below it at 724 GB before converging again. Second, W-TinyLFU

Table 6: Sizes of the four Qwen Bailian traces and the two production traces.

Trace	Requests	Block accesses
Qwen To-C	43.1 K	6.3 M
Qwen To-B	172.8 K	10.0 M
Qwen Coder	43.0 K	15.5 M
Qwen Thinking	10.8 K	3.2 M
FreeInference Trace	327.5 K	656.2 M
Chutes Trace	515.8 K	603.6 M

is nonmonotonic in cache size. On FreeInference Trace, its hit ratio rises to 0.866 at 97 GB, falls to 0.759 at 906 GB, and recovers to 0.933 at 12 TB. Thus, comparisons at a few capacities can hide both policy crossovers and nonmonotonic behavior.

B Sensitivity Analysis of Concurrency

FreeInference Trace has relatively low session concurrency, averaging about 12 concurrent sessions, and every policy except W-TinyLFU remains within roughly one point of LRU across the capacity sweep (Figure 16a). To isolate the effect of concurrency, we fix the cache at 48 GiB in HBM and 1 TiB in the pool, then synthetically vary session concurrency (Figure 18). We extract the multi-turn sessions from FreeInference Trace, truncate them where concurrency begins to collapse in the 1024-concurrency replay, and rescale their arrival times at each target concurrency. Every point therefore replays the same 73,637 requests, with concurrency as the only varied factor. Because truncation retains the earlier portion of each session, the absolute hit ratios should not be compared with §3.1; the trend across concurrency is the relevant result.

Hit ratio declines for every policy as concurrency increases, including the offline optimum. At 48 GiB, Belady falls from 0.926 at concurrency 8 to 0.713 at concurrency 32, while LRU falls from 0.898 to 0.517.

Policy differences also widen because concurrency increases the stack distance between reuses. In the HBM regime, the fraction of reuses reachable within 48 GiB falls from 92.5% at concurrency 8 to 50.1% at concurrency 32. Blocks that will be reused later in a multi-turn session thus behave as one-hit blocks within the cache horizon, making quick demotion more effective (§5.1). At concurrency 32, S3-FIFO reaches 0.573, compared with LRU’s 0.517. The pool regime shows the same effect: at concurrency 1024, only 54.1% of reuses are reachable within 1 TiB, and S3-FIFO, ARC, and LIRS overtake the other online policies. W-TinyLFU’s flatter curve in this regime is a floor effect. Its frequency filter already discards useful blocks when capacity is abundant, leaving less performance to lose as concurrency rises.

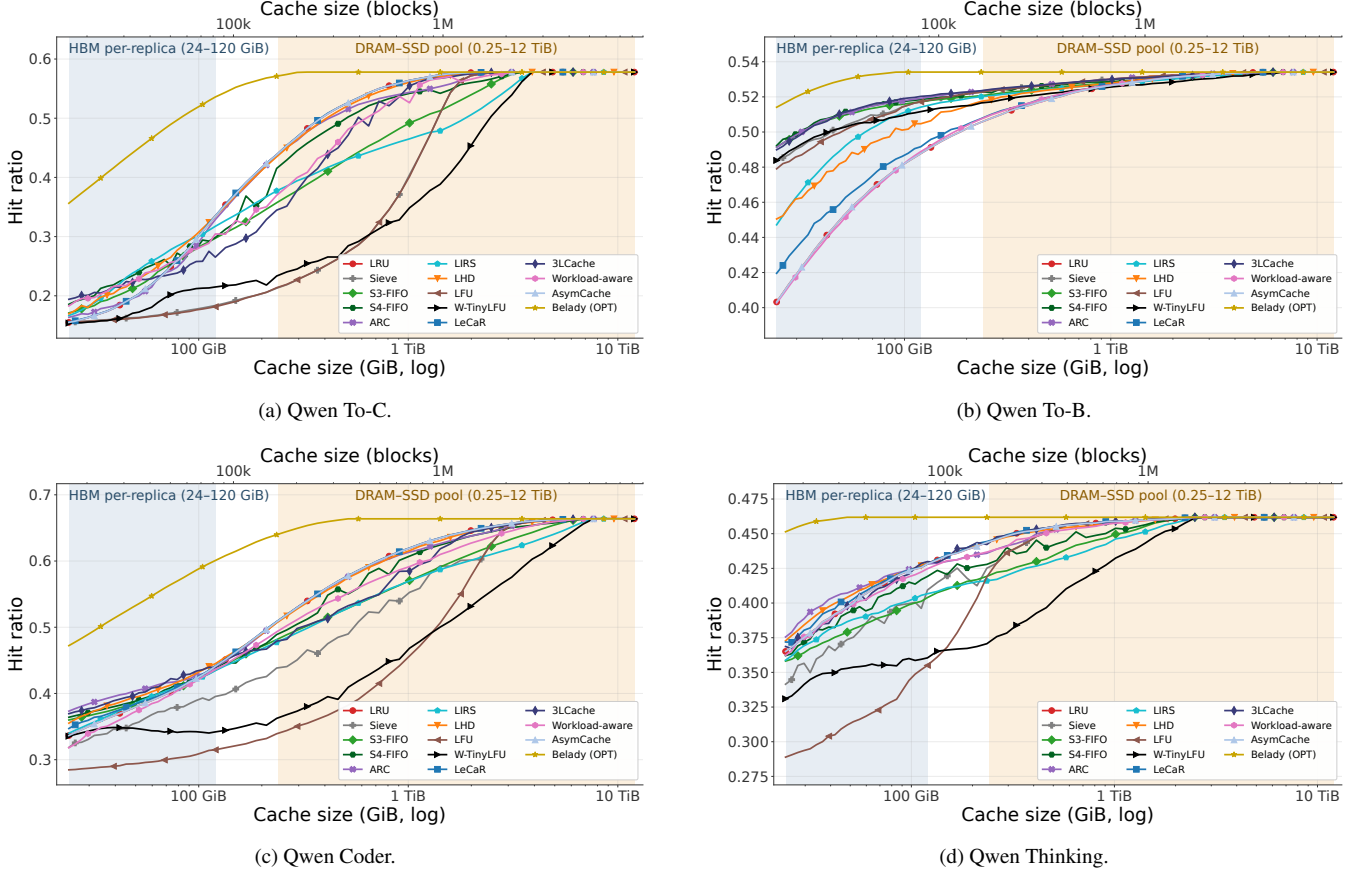


Figure 15: Block hit ratio against cache size on the four Qwen Baillan traces, over the two cache-size regimes of §3.1. Figure 11 reports the corresponding comparison at four capacities.

C Compute Savings under the Linear Cost Model

§5.2 reports compute savings under the measured Qwen3-Coder-30B cost model. Figure 19 repeats that measurement under a linear cost, the extreme case in which the attention matrix multiplication dominates and a block’s recompute cost is proportional to its depth in the prompt. The two models rank the policies identically, and every claim in §5.2 holds here with a slightly wider margin.

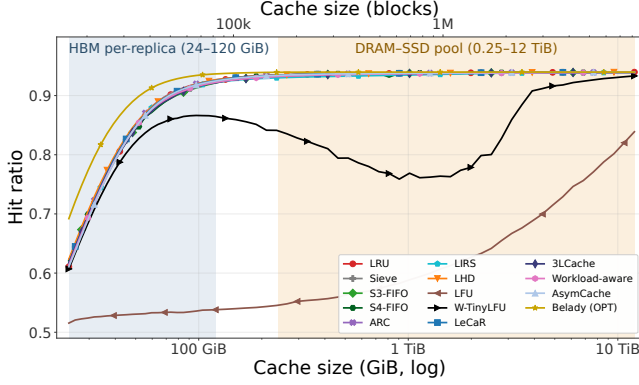
Compute-awareness pays more under the linear model, as expected: it is the model with the larger spread in per-block cost, because the measured profile adds a large position-independent term that shallow and deep blocks both pay. At 24 GiB, RandomCompute saves 0.646 against LRU’s 0.524, a margin of 12.2 points where the measured model gives 10.4, and it clears the hit-optimal Belady oracle by 4.7 points rather than 2.9. The gap between the two oracles widens the same way: BeladyCompute reaches 0.745 against Belady’s 0.599, or 14.6 points, against 12.6 under the measured model.

BeladyCompute remains a close approximation of the offline optimum: it is 0.95 points below the ILP solution at

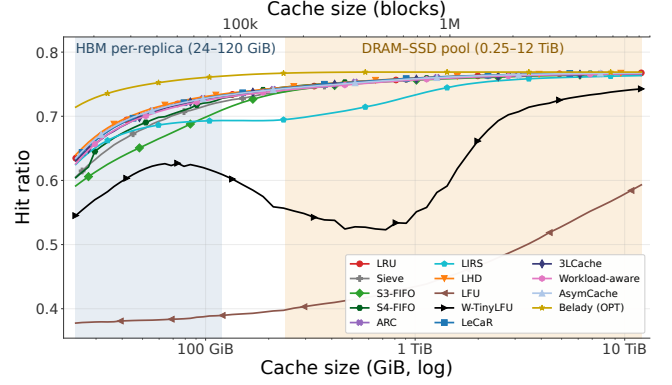
24 GiB, 0.40 at 48 GiB and 0.08 at 96 GiB, matching the measured model to within 0.1 points at every cache size. Partial-node eviction stays close to free under both: at 24 GiB it gives up 0.17 points of compute savings under the measured model and 0.03 under the linear one, so the fragmentation fix does not become more expensive when deep blocks are weighted more heavily. Past the per-replica regime the choice of cost model stops mattering: at 1 TiB every policy in the figure, both oracles included, lies within 0.2 points of the others.

D Recompute Fragmentation

Figure 20 gives the distributions behind the averages reported in §5.2. Both policies have a median of one hole per request, but RandomCompute’s tail reaches 143 holes at the 99th percentile and 281 at the maximum, compared with 2 and 16 under partial-node eviction. The total recompute volume is nearly unchanged, at 258.0 M and 257.9 M blocks, respectively. Partial-node eviction instead packs this work into longer runs: mean hole length rises from 85.5 to 742.6 blocks, while the fraction of holes no longer than two blocks falls from 56.6% to 0.8%.



(a) FreeInference Trace.



(b) Chutes Trace.

Figure 16: Block hit ratio against cache size on the two production traces of §3.2. Figure 2 reports the corresponding comparison at four capacities.

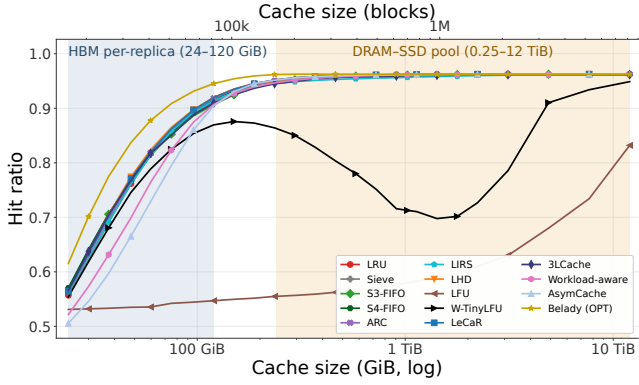


Figure 17: Block hit ratio against cache size on AgentX. Sampled at 26 capacities over the same range as Figures 15 and 16; Figure 11 reports the corresponding comparison at four capacities.

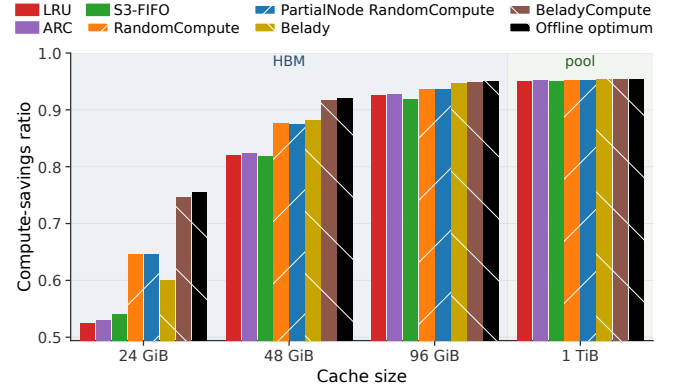


Figure 19: Compute-savings ratio on the prefix-cache trace under an idealized linear cost, where a block's recompute cost is proportional to its position in the prompt. Same policies, colors and capacities as Figure 12, which uses the measured cost model.

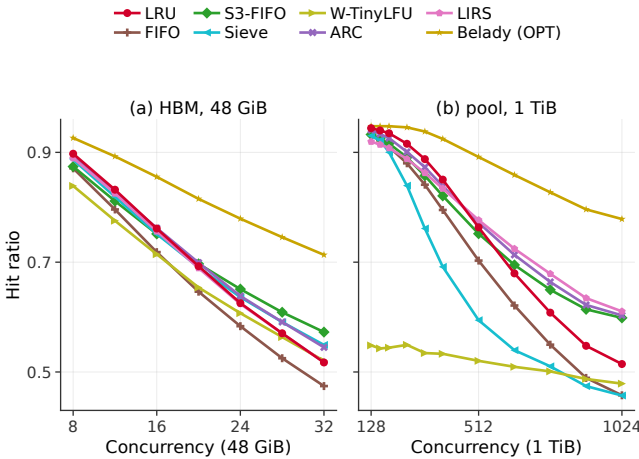


Figure 18: Hit ratio against session concurrency at a fixed cache size, on the synthetic multi-turn trace, in the per-replica HBM regime (a) and the DRAM/SSD pool regime (b); the two panels share a y axis. Every point replays the same set of requests.

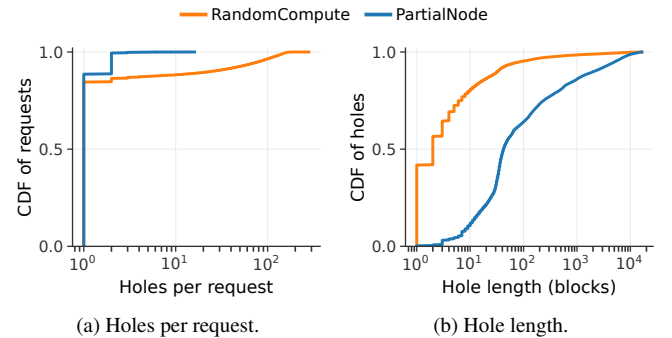


Figure 20: Effect of partial-node eviction on recompute fragmentation at 24 GiB over the 327,548 requests of FreeInference Trace. Partial-node eviction leaves fewer holes per request (a), and those holes are longer (b).